CS-338 Intro to the Theory of Computation

- Lecture #5
- **Context Free Languages**

- Prof Pietro S. Oliveto
  Department of Computer Science and Engineering
  Southern University of Science and Technology (SUSTech)
- olivetop@sustech.edu.cn
  https://faculty.sustech.edu.cn/olivetop
- Office: Room 113

18.404/6.840 Lecture 5

Last time:

- Proving languages aren't regular (Pumping Lemma)

Today: (Sipser §2.1-2.2)

- Context free grammars (CFGs) –
- Context free languages (CFLs)
- Pushdown automata (PDA)

Context Free Grammars

- We have seen *Finite State Automata* and *regular expressions*
- We showed that they are equivalent in power, they can describe many languages, but cannot describe some simple languages
- Context Free Grammars (CFGs) are a more powerful method for describing languages
- They were first used in the study of human languages
- They are important in the specification and compilation of programming languages
- Designers of compilers and interpreters for programming languages often start by obtaining a grammar for the language
- A parser extracts the meaning of a program prior to generating the compiled code or executing it
- Some methodologies even automatically generate the parser once a CFG is available

Context Free Grammars

G_1

$S \rightarrow 0S1$
$S \rightarrow R$
$R \rightarrow \varepsilon$

} (Substitution) Rules

Shorthand:

$S \rightarrow 0S1 \mid R$
$R \rightarrow \varepsilon$

In G_1:

3 rules
R,S
0,1
S

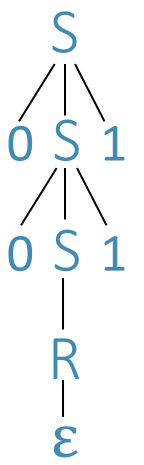
Rule: Variable → string of variables and terminals
Variables: Symbols appearing on left-hand side of rule
Terminals: Symbols appearing only on right-hand side
Start Variable: Top left symbol

Grammars generate strings

1. Write down start variable
2. Replace any variable according to a rule
   Repeat until only terminals remain
3. Result is the generated string
4. $L(G)$ is the language of all generated strings
5. We call $L(G)$ a Context Free Language.

Example of G_1 generating a string

Parse Tree of substitutions



Resulting string

S
0S1
00S11
00R11
0011 ∈ $L(G_1)$

Derivation:

$S \Rightarrow 0S1 \Rightarrow 00S11 \Rightarrow 00R11 \Rightarrow 0011$

$$L(G_1) = \{0^k 1^k \mid k \geq 0\}$$

CFG – Formal Definition

Defn: A Context Free Grammar (CFG) G is a 4-tuple (V, Σ, R, S)

- V finite set of variables
- Σ finite set of terminal symbols
- R finite set of rules **(**rule form: $V \to (V \cup \Sigma)^*$ **)**
- S start variable

For $u, v \in (V \cup \Sigma)^*$ write

1) $u \Rightarrow v$ if can go from u to v with one substitution step in G (u yields v)

2) $u \overset{*}{\Rightarrow} v$ if can go from u to v with some number of substitution steps in G (u derives v)

$u \Rightarrow u_1 \Rightarrow u_2 \Rightarrow \cdots \Rightarrow u_k = v$ is called a derivation of v from u.

If $u = S$ then it is a derivation of v.

$L(G) = \{w | \ w \in \Sigma^* \text{ and } S \overset{*}{\Rightarrow} w\}$

Defn: A is a Context Free Language (CFL) if $A = L(G)$ for some CFG G.

Designing CFGs: Techniques (1)

Design a Context Free Grammar for the language: $\{0^n1^n \mid n \geq 0\} \cup \{1^n0^n \mid n \geq 0\}$

- Many CFLs are the union of simpler CFLs
- => construct CFGs for each individual piece first with start variables $S_1, S_2, \ldots, S_k$
  (solving simpler problems is often easier than solving larger more complex ones)
- Merge the individual grammars together by combining their rules and add the rule $S \rightarrow S_1 \mid S_2 \mid \ldots \mid S_k$

$$S_1 \rightarrow 0\,S_1\,1 \mid \varepsilon$$
$$S_2 \rightarrow 1S_2\,0 \mid \varepsilon$$

For the language $\{0^n1^n \mid n \geq 0\}$
For the language $\{1^n0^n \mid n \geq 0\}$

Then add the rule for the union to get the grammar

$$S \rightarrow S_1 \mid S_2$$
$$S_1 \rightarrow 0\,S_1\,1 \mid \varepsilon$$
$$S_2 \rightarrow 1S_2\,0 \mid \varepsilon$$

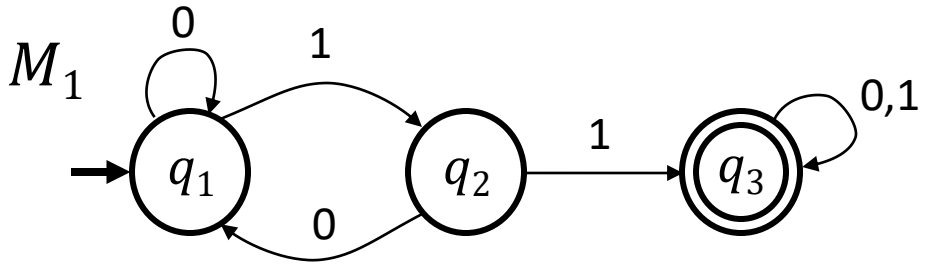
$V = \{S, S_1, S_2\}$
$\Sigma = \{0,1\}$
$R =$ the 6 rules
$S = S$

Designing CFGs: Techniques (2), DFA conversion

Designing a Context Free Grammar for a regular language is easy if you can construct a DFA for the language:

1. Make a variable R_i for each state q_i in the DFA (make R_0 the start variable where q_0 is the start state)
2. Add a rule $R_i \rightarrow a\ R_j$ for each transition $\delta(q_i, a) = q_j$ in the DFA
3. Add a rule $R_i \rightarrow \varepsilon$ if q_i is an accept state

Example



$R_1 \rightarrow 0\ R_1 \mid 1R_2$
$R_2 \rightarrow 0R_1 \mid 1R_3$
$R_3 \rightarrow 0R_3 \mid 1R_3 \mid \varepsilon$

$V = \{R_1, R_2, R_3\}$
$\Sigma =$ {0,1}
$R =$ the 7 rules
$S = R_1$

Designing CFGs: Techniques (3)

Design a Context Free Grammar for a language that has to remember an unbounded amount of information about one of the substrings

Eg, $\{0^n 1^n \mid n \geq 0\}$

You need to remember how many 0s there are before the 1s

- Use a rule of the form $R \rightarrow uRv$ where u and v are paired to keep track of the linked amounts

Eg.: $R \rightarrow 0R1$ for the above language

CFG – Example

This grammar describes a fragment of a programming language concerned with arithmetic expressions

G_2

E → E+T | T

T → T×F | F

F → (E) | a

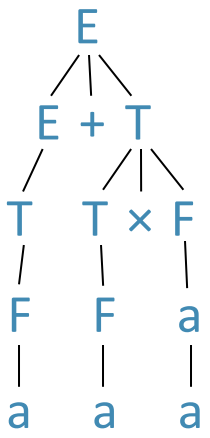
$V =$ {E, T, F}

$\Sigma =$ {+, ×, (,), a}

$R =$ the 6 rules above

$S =$ E

Parse tree



	Resulting string
E	
E+T	
T+T×F	
F+F×a	
a+a×a	$\in L(G_2)$

Generates a+a×a

Observe that the parse tree contains additional information, such as the precedence of × over + .

If a string has two different parse trees then it is derived ambiguously and we say that the grammar is ambiguous.

Ambiguity

G_2

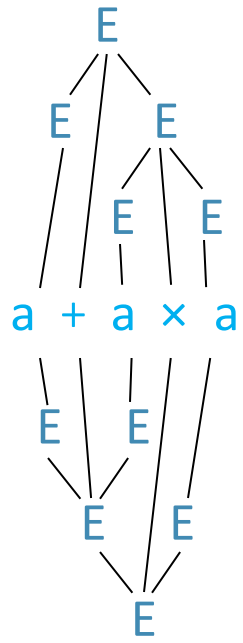
E → E+T | T

T → T×F | F

F → (E) | a

G_3

E → E+E | E×E | (E) | a



Both G_2 and G_3 generate the same language, i.e., $L(G_2) = L(G_3)$.
However G_2 is an unambiguous CFG and G_3 is ambiguous.

1. $E \Rightarrow E + E \Rightarrow a + E \Rightarrow a + E \times E \Rightarrow a + a \times E \Rightarrow a + a \times a$
2. $E \Rightarrow E \times E \Rightarrow E + E \times E \Rightarrow a + E \times E \Rightarrow a + a \times E \Rightarrow a + a \times a$

(Leftmost Derivations)

G_3 does not capture the usual precedence relations with its parse tree (it may group + before *)

Defn

"a grammar generates a string ambiguously if the string has two different parse trees" (not two different derivations – two derivations may differ in the order they replace variables, yet not the structure)

Defn(2)

"A string w is *derived ambiguously* if it has two or more different leftmost derivations. A grammar is ambiguous if it generates some string *ambiguously*."

For some ambiguous CFGs we can find an unambiguous CFG that recognises the same language. Otherwise the language is inherently ambiguous.

Chomsky Normal Form

It is often convenient to have CFGs in simplified form.
Chomsky Normal Form is useful to give algorithms for working with CFGs.

A context-free grammar is in ***Chomsky normal form*** if every rule is of the form

$$A \to BC$$
$$A \to a$$

where a is any terminal and A, B, and C are any variables—except that B and C may not be the start variable. In addition, we permit the rule $S \to \varepsilon$, where S is the start variable.

Chomsky Normal Form: Conversion

Theorem: Any Context Free Language is generated by a CFG in Chomsky Normal Form.

Proof (by construction)

Let u and v be variables or terminals

1. Add a new start variable S_0 and rule $S_0 \rightarrow S$
2. Remove all ε rules not involving the start variable:
   - If G has rule $A \rightarrow \varepsilon$ then remove. Then, for each occurrence if A on the right hand side we add new rule with that occurrence of A deleted:
     - Eg. For all rules $R \rightarrow uAv$ add rule $R \rightarrow uv$ for each occurrence of an A on the right hand side (if $R \rightarrow uAvAw$ then add rules $R \rightarrow uvAw$, $R \rightarrow uAvw$, and $R \rightarrow uvw$)
     - If rule $R \rightarrow A$ exists replace with $R \rightarrow \varepsilon$ unless that was already removed
3. Remove unit rules:
   - $A \rightarrow B$ and add the rule $A \rightarrow u$ for each rule $B \rightarrow u \in G$ (unless removed previously)
4. Convert remaining rules:
   - $A \rightarrow u_1 u_2 \ldots u_k, k \geq 3$ into $A \rightarrow u_1 A_1,\ A_1 \rightarrow u_2 A_2, \ldots, A_{k-2} \rightarrow u_{k-1} u_k$
   - Replace any terminal u_i in the preceding rule(s) with new variable U_i and add the rule $U_i \rightarrow u_i$.

Chomsky Normal Form Conversion: Example

1. The original CFG G_6 is shown on the left. The result of applying the first step to make a new start variable appears on the right.

$$S \to ASA \mid \mathtt{a}B$$
$$A \to B \mid S$$
$$B \to \mathtt{b} \mid \varepsilon$$

$$\boldsymbol{S_0 \to S}$$
$$S \to ASA \mid \mathtt{a}B$$
$$A \to B \mid S$$
$$B \to \mathtt{b} \mid \varepsilon$$

2. Remove ε-rules $B \to \varepsilon$, shown on the left, and $A \to \varepsilon$, shown on the right.

$$S_0 \to S$$
$$S \to ASA \mid \mathtt{a}B \mid \mathbf{a}$$
$$A \to B \mid S \mid \boldsymbol{\varepsilon}$$
$$B \to \mathtt{b} \mid \varepsilon$$

$$S_0 \to S$$
$$S \to ASA \mid \mathtt{a}B \mid \mathtt{a} \mid \boldsymbol{SA} \mid \boldsymbol{AS} \mid \boldsymbol{S}$$
$$A \to B \mid S \mid \varepsilon$$
$$B \to \mathtt{b}$$

3a. Remove unit rules $S \to S$, shown on the left, and $S_0 \to S$, shown on the right.

$$S_0 \to S$$
$$S \to ASA \mid \mathtt{a}B \mid \mathtt{a} \mid SA \mid AS \mid S$$
$$A \to B \mid S$$
$$B \to \mathtt{b}$$

$$S_0 \to S \mid \boldsymbol{ASA} \mid \mathbf{a}\boldsymbol{B} \mid \mathbf{a} \mid \boldsymbol{SA} \mid \boldsymbol{AS}$$
$$S \to ASA \mid \mathtt{a}B \mid \mathtt{a} \mid SA \mid AS$$
$$A \to B \mid S$$
$$B \to \mathtt{b}$$

Chomsky Normal Form Conversion: Example

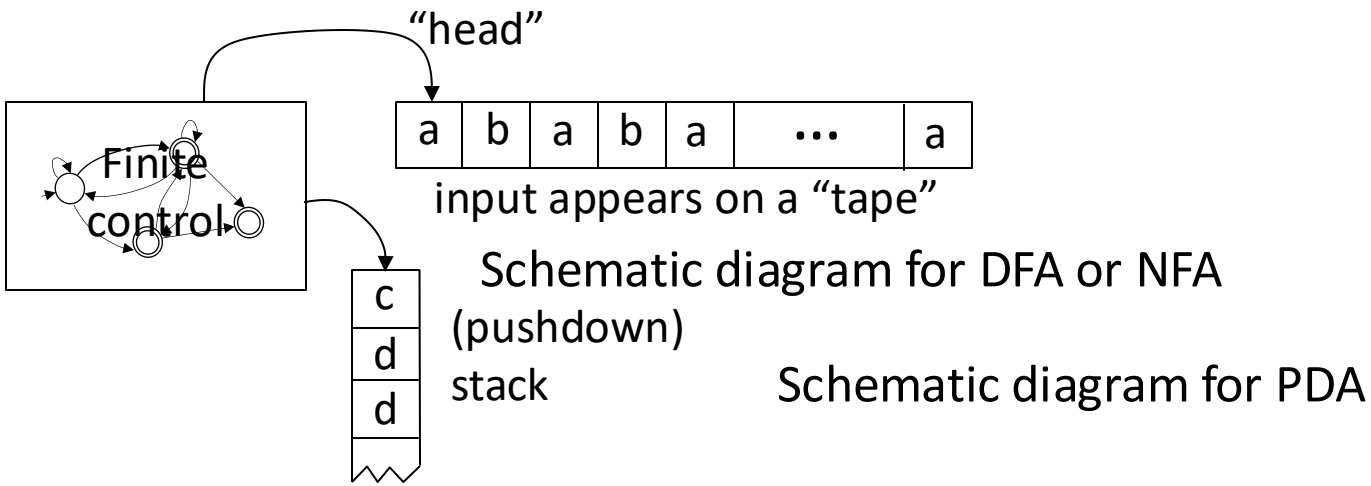
3b. Remove unit rules $A \to B$ and $A \to S$.

$$
\begin{aligned}
S_0 &\to ASA \mid \mathtt{a}B \mid \mathtt{a} \mid SA \mid AS \\
S &\to ASA \mid \mathtt{a}B \mid \mathtt{a} \mid SA \mid AS \\
A &\to B \mid S \mid \mathtt{b} \\
B &\to \mathtt{b}
\end{aligned}
\qquad
\begin{aligned}
S_0 &\to ASA \mid \mathtt{a}B \mid \mathtt{a} \mid SA \mid AS \\
S &\to ASA \mid \mathtt{a}B \mid \mathtt{a} \mid SA \mid AS \\
A &\to S \mid \mathtt{b} \mid \boldsymbol{ASA} \mid \mathtt{a}\boldsymbol{B} \mid \mathtt{a} \mid \boldsymbol{SA} \mid \boldsymbol{AS} \\
B &\to \mathtt{b}
\end{aligned}
$$

4. Convert the remaining rules into the proper form by adding additional variables and rules. The final grammar in Chomsky normal form is equivalent to G_6. (Actually the procedure given in Theorem produces several variables U_i and several rules $U_i \to \mathtt{a}$. We simplified the resulting grammar by using a single variable U and rule $U \to \mathtt{a}$.)

$$
\begin{aligned}
S_0 &\to AA_1 \mid UB \mid \mathtt{a} \mid SA \mid AS \\
S &\to AA_1 \mid UB \mid \mathtt{a} \mid SA \mid AS \\
A &\to \mathtt{b} \mid AA_1 \mid UB \mid \mathtt{a} \mid SA \mid AS \\
A_1 &\to SA \\
U &\to \mathtt{a} \\
B &\to \mathtt{b}
\end{aligned}
$$

Pushdown Automata (PDA)

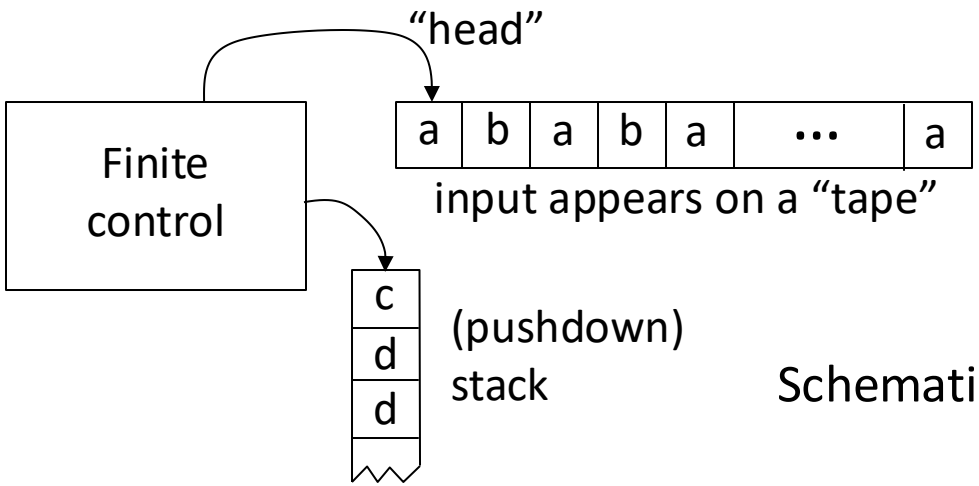


- Operates like an NFA except can write-add or read-remove symbols from the top of stack.

push pop

- The stack provides additional memory beyond the amount available in the finite control.
- Obviously only the top of the stack can be accessed.
- The stack can hold an unlimited amount of information.

Pushdown Automata (PDA): Example



Schematic diagram for PDA

Example: PDA for $D = \{0^k 1^k \mid k \geq 0\}$

The unlimited nature of the stack allows the PDA to store numbers of unbounded size

1) Read 0s from input, push onto stack until read 1.
2) Read 1s from input, while popping 0s from stack.
3) Enter accept state if stack is empty. (note: acceptance only at end of input)

PDA – Formal Definition

Defn: A Pushdown Automaton (PDA) is a 6-tuple $(Q, \Sigma, \Gamma, \delta, q_0, F)$

Σ input alphabet

Γ stack alphabet

δ: $\mathrm{Q} \times \Sigma_\varepsilon \times \Gamma_\varepsilon \to \mathcal{P}(Q \times \Gamma_\varepsilon)$ transition function

$\delta(q, \mathrm{a}, \mathrm{c}) = \{(r_1, \mathrm{d}), (r_2, \mathrm{e})\}$

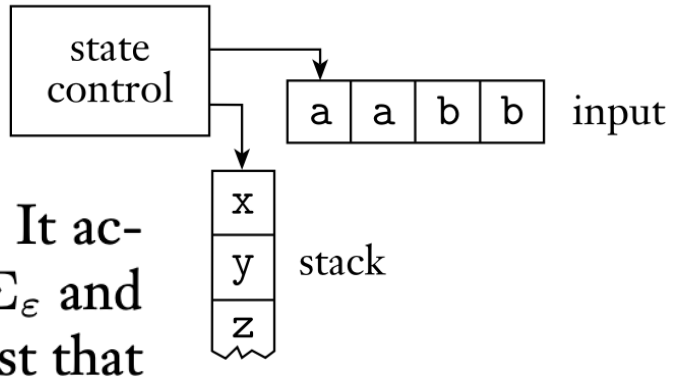
$q_0 \in Q$ start state

$F \subseteq Q$ set of accept states

Accept if some thread is in the accept state at the end of the input string.

- The machine may use different alphabets for its input string (Σ) and for its stack (Γ).
- In the domain of the transition function δ, $\mathrm{Q} \times \Sigma_\varepsilon \times \Gamma_\varepsilon$ either input or stack symbol may be ε, causing the machine to either move without reading a symbol from the input or from the stack.
- In the codomain of the transition function δ, $Q \times \Gamma_\varepsilon = (r_1, \mathrm{d})$, it moves to state r_1 and writes symbol d on the stack.
- If $Q \times \Gamma_\varepsilon = (r_1, \varepsilon)$, then it moves without writing on the stack.
- Because we allow non-determinism, each position $\mathrm{Q} \times \Sigma_\varepsilon \times \Gamma_\varepsilon$ may have several legal moves (i.e., $\mathcal{P}(Q \times \Gamma_\varepsilon)$ is in the codomain)
- The PDA can write a **$** on the empty stack to tell when it's empty

PDA – Formal Definition



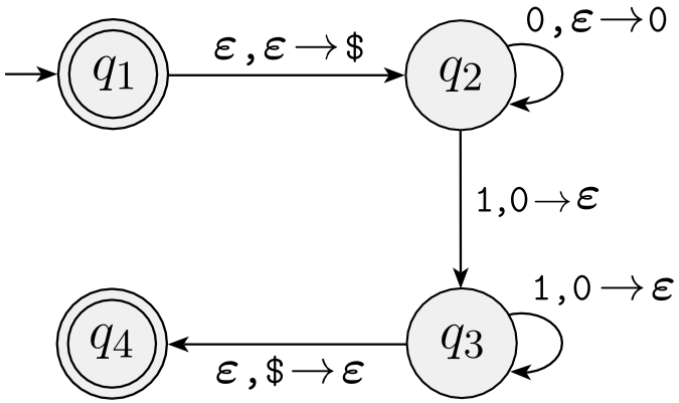
A pushdown automaton $M = (Q, \Sigma, \Gamma, \delta, q_0, F)$ computes as follows. It accepts input w if w can be written as $w = w_1 w_2 \cdots w_m$, where each $w_i \in \Sigma_\varepsilon$ and sequences of states $r_0, r_1, \ldots, r_m \in Q$ and strings $s_0, s_1, \ldots, s_m \in \Gamma^*$ exist that satisfy the following three conditions. The strings s_i represent the sequence of stack contents that M has on the accepting branch of the computation.

1. $r_0 = q_0$ and $s_0 = \boldsymbol{\varepsilon}$. This condition signifies that M starts out properly, in the start state and with an empty stack.
2. For $i = 0, \ldots, m-1$, we have $(r_{i+1}, b) \in \delta(r_i, w_{i+1}, a)$, where $s_i = at$ and $s_{i+1} = bt$ for some $a, b \in \Gamma_\varepsilon$ and $t \in \Gamma^*$. This condition states that M moves properly according to the state, stack, and next input symbol.
3. $r_m \in F$. This condition states that an accept state occurs at the input end.

PDA Definition– Example

Example: Give formal definition of a PDA
$M: (Q, \Sigma, \Gamma, \delta, q_0, F)$ for the language $\{0^n 1^n | \, n \geq 0\}$

State diagram of the PDA:



$Q = \{q_1, q_2, q_3, q_4\}$,

$\Sigma = \{0,1\}$,

$\Gamma = \{0, \$\}$,

$F = \{q_1, q_4\}$, and

δ is given by the following table, wherein blank entries signify $\emptyset$.

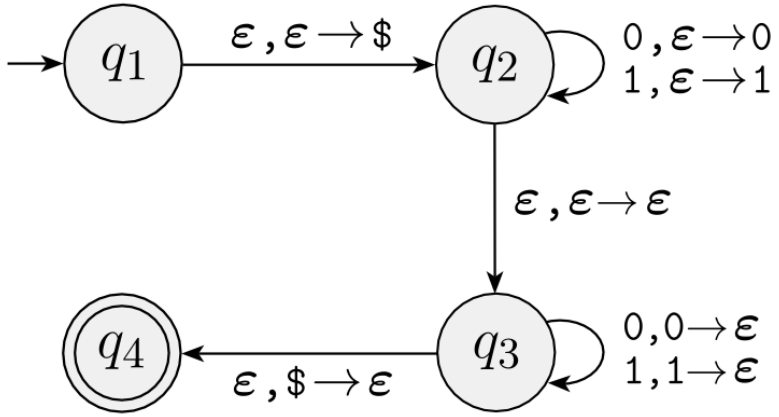
Input:	0			1			ε		
Stack:	0	\$	ε	0	\$	ε	0	\$	ε
q_1									$\{(q_2, \$)\}$
q_2			$\{(q_2, 0)\}$	$\{(q_3, \varepsilon)\}$					
q_3				$\{(q_3, \varepsilon)\}$				$\{(q_4, \varepsilon)\}$	
q_4									

PDA – Example (2)

Example: PDA for $B = \{ww^{\mathcal{R}} | \, w \in \{0,1\}^* \}$

($w^{\mathcal{R}}$ means $\mathcal{R}$ written backwards)

1) Read and push input symbols.
   Nondeterministically either repeat or go to (2).
2) Read input symbols and pop stack symbols, compare.
   If ever ≠ then thread rejects.
3) Enter accept state if stack is empty. (do in "software")



Sample input:

0	1	1	1	1	0

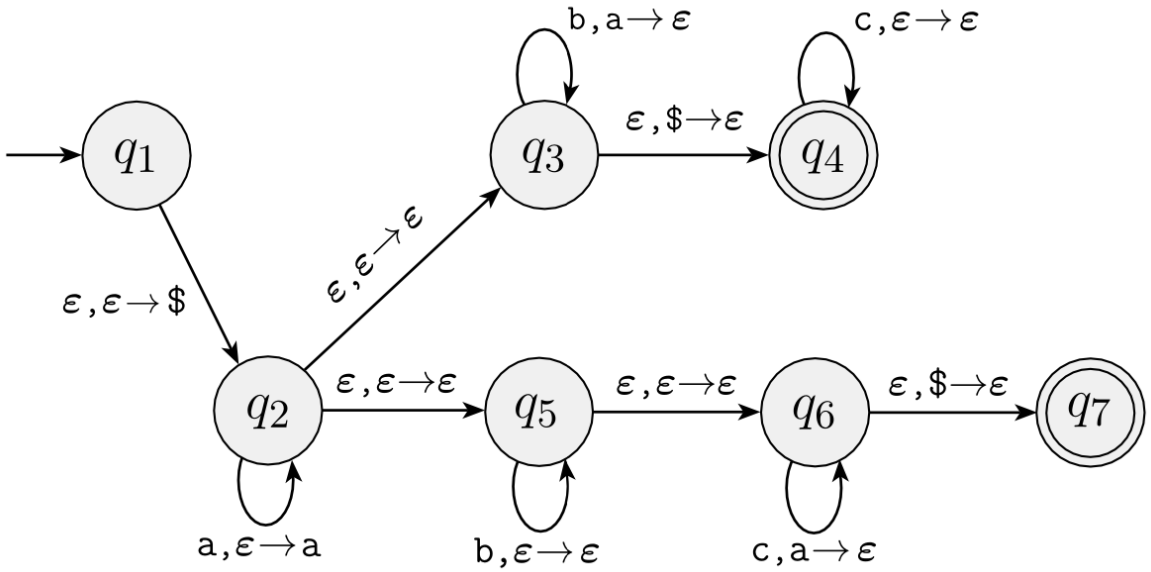
The nondeterministic forks replicate the stack.

This language requires nondeterminism.
Our PDA model is nondeterministic.

PDA – Example (3)

Example: PDA for $C = \{a^i b^j c^k \mid i = j \text{ or } i = k\}$

1) Read and push a symbols.
2) The machine doesn't know whether to match a_s with b_s or c_s: use nondeterminism to guess by using one branch per guess!
3) Enter accept state if stack is empty. (do in "software")



Sample input:

a	b	b	b	b	c

PDA – How poweful?

Which is more powerful?

- DFA or PDA?
- Regular expressions or PDA?
- NFA or PDA?
- Regular expressions or CFGs?
- CFGs or PDAs?

Next: Show that CFGs and PDAs are equivalent

- Both are capable of describing Context Free Languages
- We show how to convert any CFG in a PDA that recognises the same language and viceversa
- Recall: CFL are any language that can be described by a CFG.

Quick review of today

1. Defined Context Free Grammars (CFGs) and Context Free Languages (CFLs)
2. Defined Pushdown Automata(PDAs)